

COMPUTER NETWORKS

LAB MANUAL

B.TECH II-YEAR II-SEM (R25 REGULATION)
SUBJECT CODE: B25CS214



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

VAAGDEVI ENGINEERING COLLEGE

(An Autonomous Institution | Affiliated to JNTU Hyderabad)

Accredited by NAAC with 'A+' Grade, Certified by ISO9001:2015

Approved by AICTE, New Delhi & Affiliated to JNTUH, Hyderabad

BOLLIKUNTA, KHILA WARANGAL (MANDAL), WARANGAL 506005, TELANGANA, INDIA

EDUCATIONAL OBJECTIVES AND OUTCOMES

B. Tech - Program Educational Objectives (PEO's)

The Program aims to:

- **PEO1:** To apply fundamental concepts of data communication and computer networks to design and build reliable networking solutions for real-world problems.
- **PEO2:** To demonstrate proficiency in understanding and implementing protocols across various layers of the OSI and TCP/IP reference models.
- **PEO3:** To exhibit problem-solving skills and analytical thinking in designing IP addressing schemes, subnetting, and routing mechanisms.
- **PEO4:** To adapt to evolving technologies and tools such as network simulators and packet sniffers for continuous learning and professional growth.
- **PEO5:** To work effectively in teams and communicate technical networking architectures clearly while developing software and hardware network solutions.

B.Tech - Program Outcomes (Po's)

- **PO1: Network Fundamentals** Ability to understand and analyze network topologies, transmission media, and the core functionalities of OSI and TCP/IP layers.
- **PO2: Protocol Analysis** Ability to analyze and implement various protocols at the data link, network, and transport layers, including error detection, MAC, and routing algorithms.
- **PO3: IP Addressing and Subnetting** Ability to design and configure logical addressing schemes using IPv4 and IPv6 for efficient network management.
- **PO4: Network Security and Applications** Ability to understand basic cryptographic principles, network security concepts, and application layer protocols like HTTP, DNS, and SMTP.
- **PO5: Network Troubleshooting** Ability to identify, diagnose, and resolve network performance issues and data transmission problems.
- **PO6: Use of Development Tools** Ability to use modern network simulation and packet analysis tools like Cisco Packet Tracer, NS2, or Wireshark for configuring and testing networks.
- **PO7: Teamwork and Communication** Ability to work collaboratively in teams to design, implement, and document computer network architectures.
- **PO8: Lifelong Learning** Ability to continuously update knowledge in emerging networking technologies like Software Defined Networking (SDN), Wireless Networks, and IoT.

II Year B.Tech. CSE(DS) II-Sem

Course Objectives

1. To understand the working principle of various communication protocols.
2. To understand the network simulator environment and visualize a network topology and observe its performance
3. To analyze the traffic flow and the contents of protocol frames

Course Outcomes

1. Implement data link layer framing methods
2. Analyze error detection and error correction codes.
3. Implement and analyze routing and congestion issues in network design.
4. Implement Encoding and Decoding techniques used in presentation layer
5. To be able to work with different network tools

List of Experiments

1. Implement the data link layer framing methods such as character, character-stuffing and bit stuffing.
2. Write a program to compute CRC code for the polynomials CRC-12, CRC-16 and CRC CCIP
3. Develop a simple data link layer that performs the flow control using the sliding window protocol, and loss recovery using the Go-Back-N mechanism.
4. Implement Dijkstra's algorithm to compute the shortest path through a network
5. Take an example subnet of hosts and obtain a broadcast tree for the subnet.
6. Implement distance vector routing algorithm for obtaining routing tables at each node.
7. Implement data encryption and data decryption
8. Write a program for congestion control using Leaky bucket algorithm.
9. Write a program for frame sorting technique used in buffers.
10. Wireshark
 - i. Packet Capture Using Wire shark
 - ii. Starting Wire shark
 - iii. Viewing Captured Traffic
 - iv. Analysis and Statistics & Filters.
11. How to run Nmap scan
12. Operating System Detection using Nmap
13. Do the following using NS2 Simulator
 - i. NS2 Simulator-Introduction
 - ii. Simulate to Find the Number of Packets Dropped
 - iii. Simulate to Find the Number of Packets Dropped by TCP/UDP
 - iv. Simulate to Find the Number of Packets Dropped due to Congestion
 - v. Simulate to Compare Data Rate & Throughput.
 - vi. Simulate to Plot Congestion for Different Source/Destination
 - vii. Simulate to Determine the Performance with respect to Transmission of Packets

1. Implement the data link layer framing methods such as character, character-stuffing and bit stuffing.

Program 1(a): Character Stuffing

Code:

```
// CHARACTER STUFFING PROGRAM
#include<stdio.h>
#include<conio.h>
#include<string.h>
#include<process.h>

void main()
{
    int i=0,j=0,n,pos;
    char a[20],b[50],ch;
    clrscr();
    printf("enter string\n");
    scanf("%s",&a);
    n=strlen(a);
    printf("enter position\n");
    scanf("%d",&pos);
    if(pos>n)
    {
        printf("invalid position, Enter again :");
        scanf("%d",&pos);
    }
    printf("enter the character\n");
    scanf("%s",&ch);
    b[0]='d'; b[1]='l'; b[2]='e'; b[3]='s'; b[4]='t'; b[5]='x';
    j=6;
    while(i<n)
    {
        if(i==pos-1)
        {
            b[j]='d'; b[j+1]='l'; b[j+2]='e'; b[j+3]=ch;
            b[j+4]='d'; b[j+5]='l'; b[j+6]='e';
            j=j+7;
        }
        if(a[i]=='d' && a[i+1]=='l' && a[i+2]=='e')
        {
            b[j]='d'; b[j+1]='l'; b[j+2]='e';
            j=j+3;
        }
        b[j]=a[i];
        i++;
        j++;
    }
    b[j]='d'; b[j+1]='l'; b[j+2]='e'; b[j+3]='e'; b[j+4]='t'; b[j+5]='x';
    b[j+6]='\0';
    printf("\nframe after stuffing:\n");
    printf("%s",b);
    getch();
}
```

Output:

```
enter string
hello
enter position
3
enter the character
x

frame after stuffing:
dlestxheldlexdleloetx
```

Program 1(b): Bit Stuffing

Code:

```
// BIT STUFFING PROGRAM
#include<stdio.h>
#include<conio.h>
#include<string.h>

void main()
{
    int a[20],b[30],i,j,k,count,n;
    clrscr();
    printf("Enter frame length:");
    scanf("%d",&n);
    printf("Enter input frame (0's & 1's only):");
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);
    i=0;
    count=1;
    j=0;
    while(i<n)
    {
        if(a[i]==1)
        {
            b[j]=a[i];
            for(k=i+1;a[k]==1 && k<n && count<5;k++)
            {
                j++;
                b[j]=a[k];
                count++;
                if(count==5)
                {
                    j++;
                    b[j]=0;
                }
                i=k;
            }
        }
        else
        {
            b[j]=a[i];
        }
        i++;
        j++;
    }
    printf("After stuffing the frame is:");
    for(i=0;i<j;i++)
        printf("%d",b[i]);
    getch();
}
```

Output:

```
Enter frame length:10  
Enter input frame (0's & 1's only):1 1 1 1 1 1 0 0 1 0  
After stuffing the frame is:111110110010
```

2. Write a program to compute CRC code for the polynomials CRC-12, CRC-16 and CRC CCIP

Code:

```
// CYCLIC REDUNDANCY CHECK PROGRAM
#include<stdio.h>
#include<conio.h>

void remainder(int fr[]);
int gen[4],genl,frl,rem[4];

void main()
{
    int i,j,fr[8],dupfr[11],recfr[11],tlen,flag;
    clrscr();
    frl=8;
    genl=4;
    printf("enter frame:");
    for(i=0;i<frl;i++)
    {
        scanf("%d",&fr[i]);
        dupfr[i]=fr[i];
    }
    printf("enter generator:");
    for(i=0;i<genl;i++)
        scanf("%d",&gen[i]);

    tlen=frl+genl-1;
    for(i=frl;i<tlen;i++)
    {
        dupfr[i]=0;
    }
    remainder(dupfr);
    for(i=0;i<frl;i++)
    {
        recfr[i]=fr[i];
    }
    for(i=frl,j=1;j<genl;i++,j++)
    {
        recfr[i]=rem[j];
    }
    remainder(recfr);
    flag=0;
    for(i=0;i<4;i++)
    {
        if(rem[i]!=0) flag++;
    }
    if(flag==0)
    {
        printf("frame received correctly");
    }
    else
    {
        printf("the received frame is wrong");
    }
}
```



```

    getch();
}

void remainder(int fr[])
{
    int k,k1,i,j;
    for(k=0;k<frl;k++)
    {
        if(fr[k]==1)
        {
            k1=k;
            for(i=0,j=k;i<genl;i++,j++)
            {
                rem[i]=fr[j]^gen[i];
            }
            for(i=0;i<genl;i++)
            {
                fr[k1]=rem[i];
                k1++;
            }
        }
    }
}

```

Output:

```

enter frame:1 0 0 1 1 0 1 0
enter generator:1 1 0 1
frame received correctly

```

3. Develop a simple data link layer that performs the flow control using the sliding window protocol, and loss recovery using the Go-Back-N mechanism.

Code:

```
#include<stdio.h>

int main()
{
    int w,i,f,frames[50];

    printf("Enter window size: ");
    scanf("%d",&w);

    printf("\nEnter number of frames to transmit: ");
    scanf("%d",&f);

    printf("\nEnter %d frames: ",f);

    for(i=1;i<=f;i++)
        scanf("%d",&frames[i]);

    printf("\nWith sliding window protocol the frames will be sent in the following manner (assuming no corruption of frames)\n\n");
    printf("After sending %d frames at each stage sender waits for acknowledgement sent by the receiver\n\n",w);

    for(i=1;i<=f;i++)
    {
        if(i%w==0)
        {
            printf("%d\n",frames[i]);
            printf("Acknowledgement of above frames sent is received by sender\n\n");
        }
        else
            printf("%d ",frames[i]);
    }

    if(f%w!=0)
        printf("\nAcknowledgement of above frames sent is received by sender\n");

    return 0;
}
```

Output:

```
Enter window size: 3
Enter number of frames to transmit: 6
Enter 6 frames: 1 2 3 4 5 6
With sliding window protocol the frames will be sent in the following manner (assuming no corruption of frames)
After sending 3 frames at each stage sender waits for acknowledgement sent by the receiver
1 2 3
Acknowledgement of above frames sent is received by sender
4 5 6
Acknowledgement of above frames sent is received by sender
```

4. Implement Dijkstra's algorithm to compute the shortest path through a network

Code:

```
#include<stdio.h>

int main()
{
    int path[5][5],i,j,min,a[5][5],p,st=1,ed=5,stp,edp,t[5],index;
    printf("enter the cost matrix\n");
    for(i=1;i<=5;i++)
        for(j=1;j<=5;j++)
            scanf("%d",&a[i][j]);
    printf("enter the paths\n");
    scanf("%d",&p);
    printf("enter possible paths\n");
    for(i=1;i<=p;i++)
        for(j=1;j<=5;j++)
            scanf("%d",&path[i][j]);
    for(i=1;i<=p;i++)
    {
        t[i]=0;
        stp=st;
        for(j=1;j<=5;j++)
        {
            edp=path[i][j+1];
            t[i]=t[i]+a[stp][edp];
            if(edp==ed) break;
            else
                stp=edp;
        }
    }
    min=t[st];
    index=st;
    for(i=1;i<=p;i++)
    {
        if(min>t[i])
        {
            min=t[i];
            index=i;
        }
    }
    printf("minimum cost %d",min);
    printf("\n minimum cost path ");
    for(i=1;i<=5;i++)
    {
        printf("--> %d",path[index][i]);
        if(path[index][i]==ed) break;
    }
    return 0;
}
```

Output:

```
enter the cost matrix
0 2 9 5 0
0 0 3 0 7
0 0 0 4 0
0 0 0 0 1
0 0 0 0 0
enter the paths
2
enter possible paths
1 2 3 4 5
1 2 4 5 0
minimum cost 3
  minimum cost path --> 1--> 2--> 4--> 5
```

5. Take an example subnet of hosts and obtain a broadcast tree for the subnet.

Code:

```
#include<stdio.h>

int a[10][10],n;

void adj(int k);

int main()
{
    int i,j,root;
    printf("Enter no.of nodes:");
    scanf("%d",&n);
    printf("Enter adjacent matrix\n");
    for(i=1;i<=n;i++)
        for(j=1;j<=n;j++)
        {
            printf("Enter connecting of %d>%d::",i,j);
            scanf("%d",&a[i][j]);
        }
    printf("Enter root node:");
    scanf("%d",&root);
    adj(root);
    return 0;
}

void adj(int k)
{
    int i,j;
    printf("Adjacent node of root node::\n");
    printf("%d\n",k);
    for(j=1;j<=n;j++)
    {
        if(a[k][j]==1 || a[j][k]==1)
            printf("%d\t",j);
    }
    printf("\n");
    for(i=1;i<=n;i++)
    {
        if((a[k][j]==0) && (a[i][k]==0) && (i!=k))
            printf("%d",i);
    }
}
```

Output:

```
Enter no.of nodes:3
Enter adjacent matrix
Enter connecting of 1>1::1
Enter connecting of 1>2::1
Enter connecting of 1>3::0
Enter connecting of 2>1::1
Enter connecting of 2>2::0
Enter connecting of 2>3::1
Enter connecting of 3>1::0
Enter connecting of 3>2::1
Enter connecting of 3>3::0
Enter root node:3
Adjacent node of root node::
3
2
1
```

6. Implement distance vector routing algorithm for obtaining routing tables at each node.

Code:

```
#include<stdio.h>

struct node
{
    unsigned dist[20];
    unsigned from[20];
}rt[10];

int main()
{
    int dmat[20][20];
    int n,i,j,k,count=0;
    printf("\nEnter the number of nodes : ");
    scanf("%d",&n);
    printf("\nEnter the cost matrix :\n");
    for(i=0;i<n;i++)
        for(j=0;j<n;j++)
        {
            scanf("%d",&dmat[i][j]);
            dmat[i][i]=0;
            rt[i].dist[j]=dmat[i][j];
            rt[i].from[j]=j;
        }
    do
    {
        count=0;
        for(i=0;i<n;i++)
            for(j=0;j<n;j++)
                for(k=0;k<n;k++)
                    if(rt[i].dist[j]>dmat[i][k]+rt[k].dist[j])
                    {
                        rt[i].dist[j]=rt[i].dist[k]+rt[k].dist[j];
                        rt[i].from[j]=k;
                        count++;
                    }
    }while(count!=0);
    for(i=0;i<n;i++)
    {
        printf("\n\nState value for router %d is \n",i+1);
        for(j=0;j<n;j++)
        {
            printf("\t\nnode %d via %d\n",j+1,rt[i].from[j]+1,rt[i].dist[j]);
        }
        printf("\n\n");
    }
}
```


Output:

```
Enter the number of nodes : 3
Enter the cost matrix :
0 2 9
2 0 3
9 3 0
```

```
State value for router 1 is
node 1 via 1 Distance0
node 2 via 2 Distance2
node 3 via 2 Distance5
```

```
State value for router 2 is
node 1 via 1 Distance2
node 2 via 2 Distance0
node 3 via 3 Distance3
```

```
State value for router 3 is
node 1 via 2 Distance5
node 2 via 2 Distance3
node 3 via 3 Distance0
```

7. Implement data encryption and data decryption

Code:

```
#include <stdio.h>

int main() {
    int i, x;
    char str[100];

    printf("\nPlease enter a string:\t");
    gets(str);

    printf("\nPlease choose following options:\n");
    printf("1 = Encrypt the string.\n");
    printf("2 = Decrypt the string.\n");
    scanf("%d", &x);

    //using switch case statements
    switch(x)
    {
    case 1:
        for(i = 0; (i < 100 && str[i] != '\0'); i++)
            str[i] = str[i] + 3; //the key for encryption is 3 that is added to ASCII
value

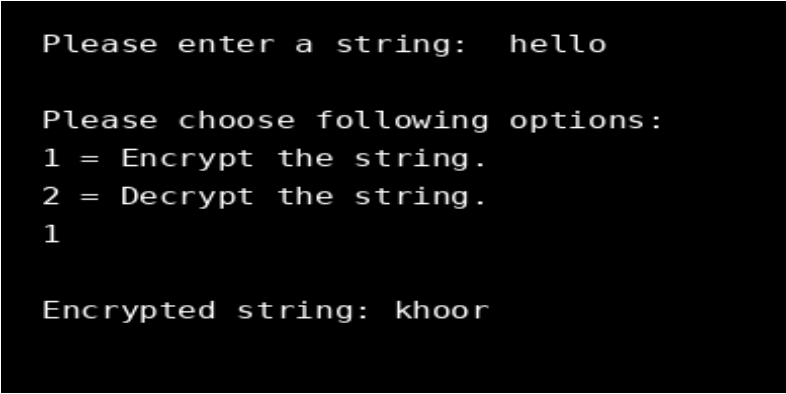
        printf("\nEncrypted string: %s\n", str);
        break;

    case 2:
        for(i = 0; (i < 100 && str[i] != '\0'); i++)
            str[i] = str[i] - 3; //the key for encryption is 3 that is subtracted to
ASCII value

        printf("\nDecrypted string: %s\n", str);
        break;

    default:
        printf("\nError\n");
    }
    return 0;
}
```

Output 1: Encryption



```
Please enter a string:  hello

Please choose following options:
1 = Encrypt the string.
2 = Decrypt the string.
1

Encrypted string: khood
```

Output 2: Decryption

```
Please enter a string:  koor
```

```
Please choose following options:
```

```
1 = Encrypt the string.
```

```
2 = Decrypt the string.
```

```
2
```

```
Decrypted string: hello
```

8. Write a program for congestion control using Leaky bucket algorithm.

Code:

```
#include<stdio.h>

int main(){
    int incoming, outgoing, buck_size, n, store = 0;
    printf("Enter bucket size, outgoing rate and no of inputs: ");
    scanf("%d %d %d", &buck_size, &outgoing, &n);

    while (n != 0) {
        printf("Enter the incoming packet size : ");
        scanf("%d", &incoming);
        printf("Incoming packet size %d\n", incoming);
        if (incoming <= (buck_size - store)){
            store += incoming;
            printf("Bucket buffer size %d out of %d\n", store, buck_size);
        } else {
            printf("Dropped %d no of packets\n", incoming - (buck_size - store));
            printf("Bucket buffer size %d out of %d\n", store, buck_size);
            store = buck_size;
        }
        store = store - outgoing;
        printf("After outgoing %d packets left out of %d in buffer\n", store,
            buck_size);
        n--;
    }
}
```

Output:

```
Enter bucket size, outgoing rate and no of inputs: 10 2 3
Enter the incoming packet size : 4
Incoming packet size 4
Bucket buffer size 4 out of 10
After outgoing 2 packets left out of 10 in buffer
Enter the incoming packet size : 6
Incoming packet size 6
Bucket buffer size 8 out of 10
After outgoing 6 packets left out of 10 in buffer
Enter the incoming packet size : 8
Incoming packet size 8
Dropped 4 no of packets
Bucket buffer size 6 out of 10
After outgoing 8 packets left out of 10 in buffer
```

9. Write a program for frame sorting technique used in buffers

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct frame {
    int sno;
    char msg[15];
    int flag;
};

int main() {
    int i,j,n,r,k;
    printf("enter no of frames\n");
    scanf("%d",&n);
    struct frame fr[n];
    int s[n];
    for(i=0;i<n;i++)
    {
        s[i]=-1;
        fr[i].sno=-1;
    }
    printf("enter the message \n");
    for(i=0;i<n;i++)
    {
        scanf("%s",fr[i].msg);
        fr[i].sno=i;
    }
    for(j=0;j<n;j++)
    {
        r=rand()%n;
        if(s[r]==-1)
        {
            fr[j].flag=r;
            s[r]=1;
        }
        else if(s[r]==1)
        {
            for(k=0;k<n;k++){
                r=k;
                if(s[r]==-1)
                {
                    fr[j].flag=r;
                    s[r]=1;
                    break;
                }
            }
        }
    }
    printf("arrived frame are:\n");
    printf("\n sno \t msg  \n");
    for(i=0;i<n;i++)
```

```

{
    for(j=0;j<n;j++)
        if(fr[j].flag==i)
        {
            printf("%d\t%s",fr[j].sno,fr[j].msg);
            printf("\n");
        }
    }
for(i=0;i<n;i++)
{
    for(j=0;j<n-1;j++)
    {
        if(fr[j].sno>fr[j+1].sno)
        {
            struct frame temp;
            temp=fr[j];
            fr[j]=fr[j+1];
            fr[j+1]=temp;
        }
    }
}
printf("after sorting arrived frames are\n");
printf("\n sno \t msg \n");
for(i=0;i<n;i++)
{
    printf("%d\t%s",fr[i].sno,fr[i].msg);
    printf("\n");
}
return 0;
}

```

Output:

```

enter no of frames
4
enter the message
hello
world
test
foo
arrived frame are:

sno    msg
3      foo
2      test
1      world
0      hello
after sorting arrived frames are

sno    msg
0      hello
1      world
2      test
3      foo

```

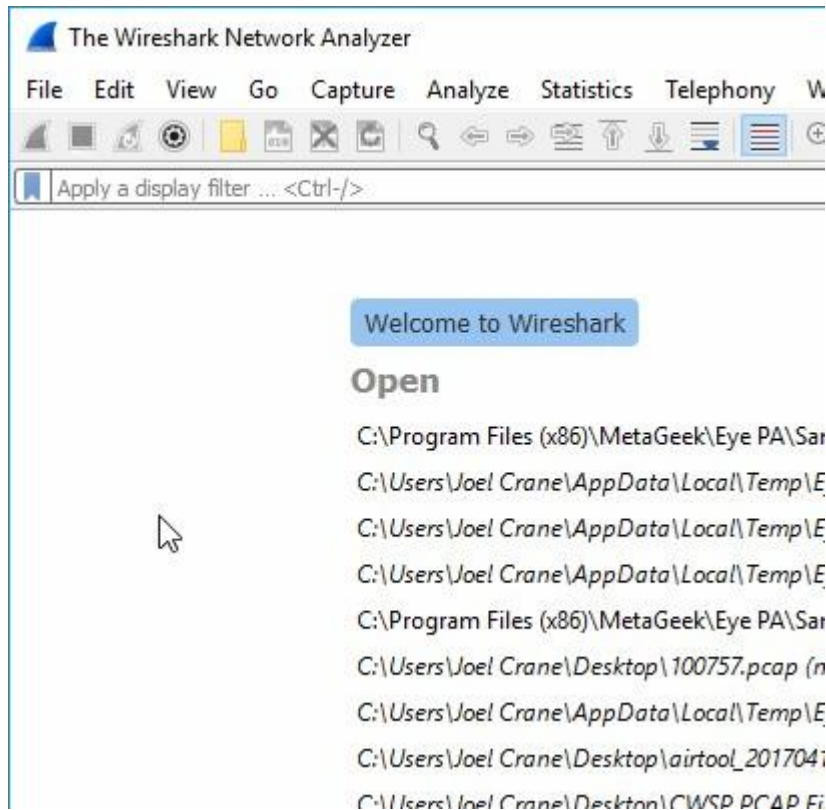
10. Wireshark

i. Packet Capture Using Wireshark

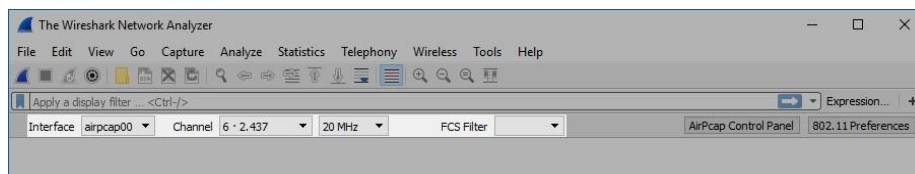
- ii. Starting Wireshark
- iii. Viewing Captured Traffic
- iv. Analysis and Statistics & Filters

Set up the Packet Capture

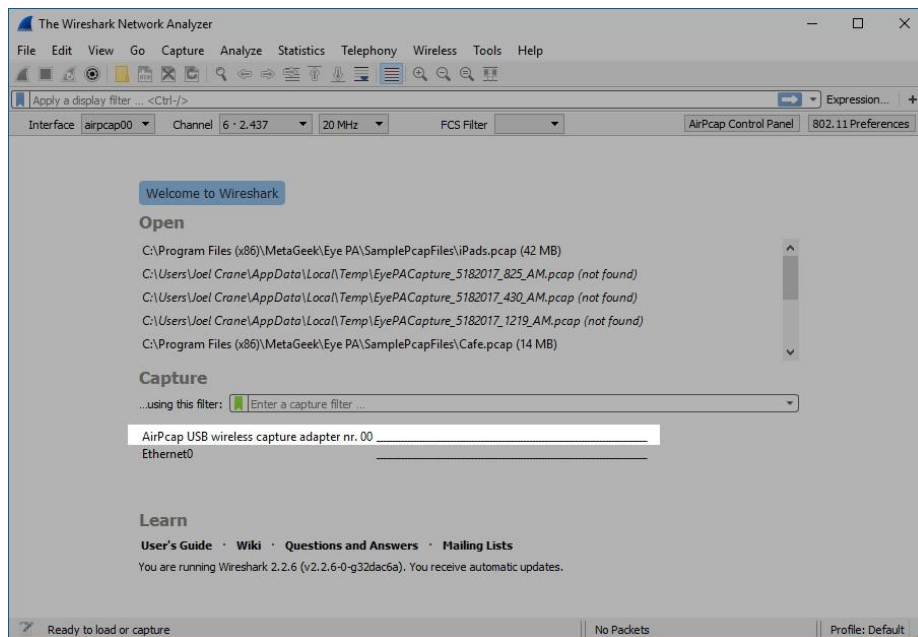
1. Click **View > Wireless Toolbar**. The Wireless Toolbar will appear just below the Main toolbar.



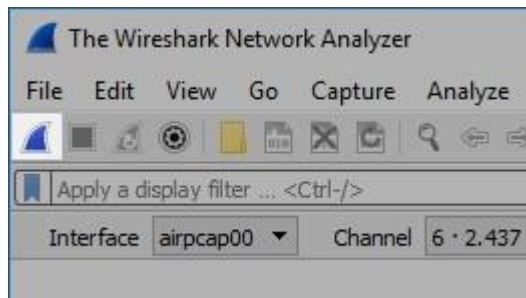
2. Use the **Wireless Toolbar** to configure the desired channel and channel width.



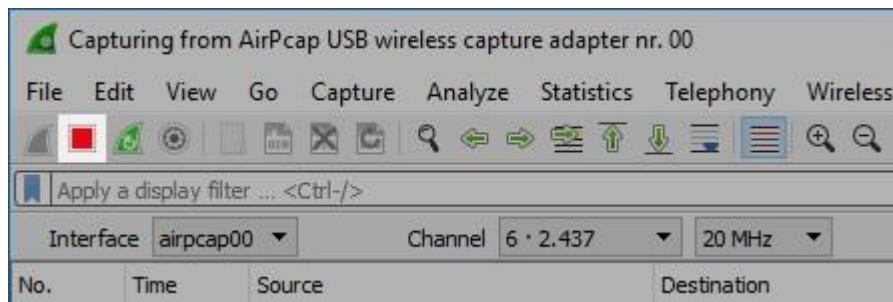
3. Under **Capture**, click on AirPcap USB wireless capture adapter to select the capture interface.
Note: If the AirPcap isn't listed, press F5 to refresh the list of available packet capture interfaces.
Note: The AirPcap has been discontinued by RiverBed and is 802.11n only.



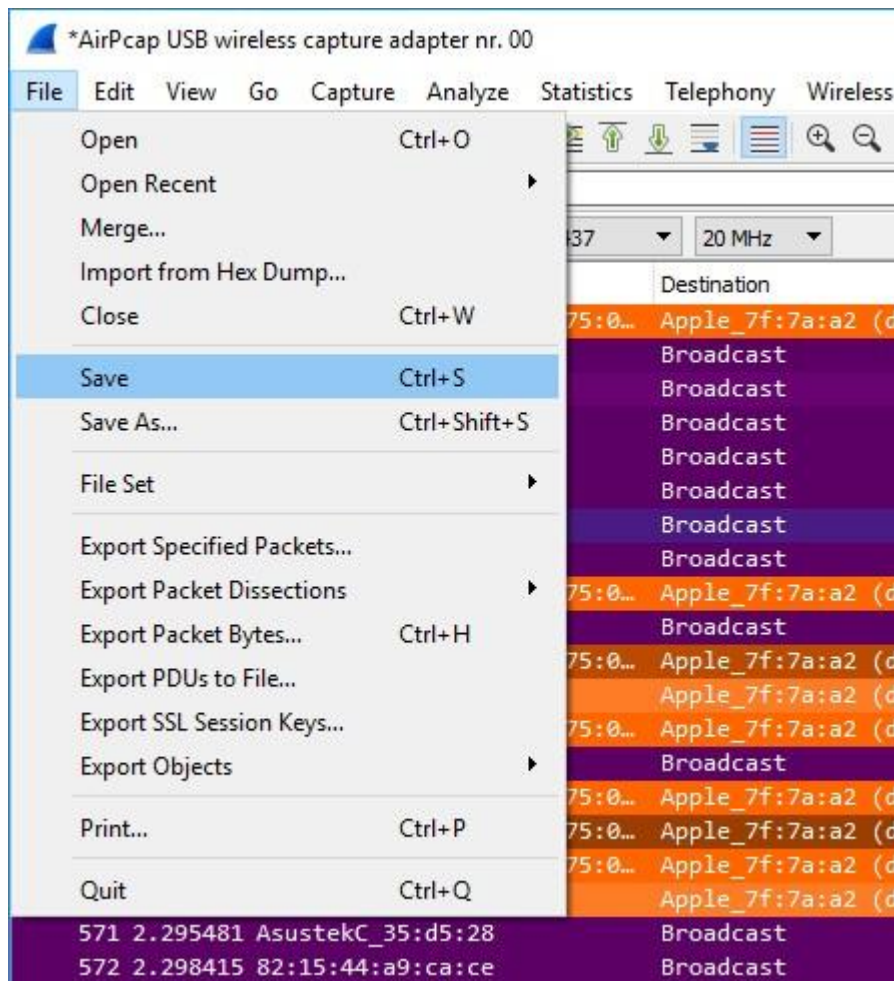
4. Click the Start Capture button to begin the capture.



5. When you are finished capturing, click the Stop button.

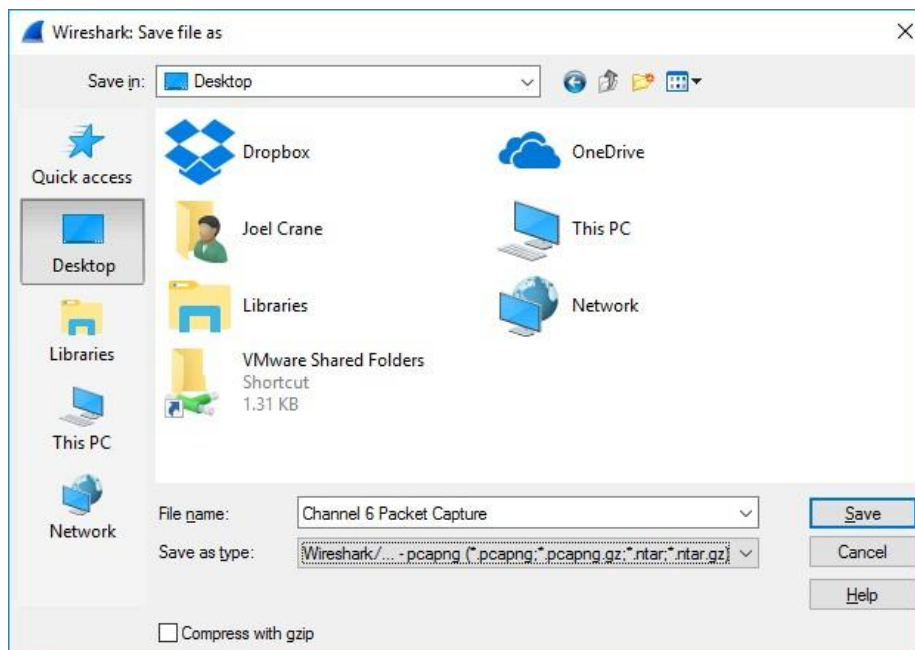


Saving the Capture



2. Name the file, and click Save.

Note: .Pcap and .Pcap-ng are good filetypes to use for the capture if you plan to use Eye P.A. to open the capture.



3. Eye P.A. can now open the capture file.

11. How to Run Nmap Scan

Steps to Run Nmap Scan

1. **Download the Nmap installer.** This can be found for free from the developer's website. It is highly recommended that you download directly from the developer to avoid any potential viruses or fake files. Downloading the Nmap installer includes Zenmap, the graphical interface for Nmap which makes it easy for newcomers to perform scans without having to learn command lines. The Zenmap program is available for Windows, Linux, and Mac OS X.
2. **Install Nmap.** Run the installer once it is finished downloading. You will be asked which components you would like to install. In order to get the full benefit of Nmap, keep all of these checked. Nmap will not install any adware or spyware.
3. **Run the "Nmap - Zenmap" GUI program.** If you left your settings at default during installation, you should be able to see an icon for it on your desktop. If not, look in your Start menu. Opening Zenmap will start the program.
4. **Enter in the target for your scan.** The Zenmap program makes scanning a fairly simple process. The first step to running a scan is choosing your target. You can enter a domain (example.com), an IP address (127.0.0.1), a network (192.168.1.0/24), or a combination of those.
 - Depending on the intensity and target of your scan, running an Nmap scan may be against the terms of your internet service provider, and may land you in hot water. Always check your local laws and your ISP contract before performing Nmap scans on targets other than your own network.
5. **Choose your Profile.** Profiles are preset groupings of modifiers that change what is scanned. The profiles allow you to quickly select different types of scans without having to type in the modifiers on the command line.
 - **Intense scan** - A comprehensive scan. Contains OS detection, version detection, script scanning, traceroute, and has aggressive scan timing. This is considered an intrusive scan.
 - **Ping scan** - This scan simply detects if the targets are online, it does not scan any ports.
 - **Quick scan** - This is quicker than a regular scan due to aggressive timing and only scanning select ports.
 - **Regular scan** - This is the standard Nmap scan without any modifiers. It will return ping and return open ports on the target.
6. **Click Scan to start scanning.** The active results of the scan will be displayed in the Nmap Output tab. The time the scan takes will depend on the scan profile you chose, the physical distance to the target, and the target's network configuration.
7. **Read your results.** Once the scan is finished, you'll see the message "Nmap done" at the bottom of the Nmap Output tab. You can now check your results, depending on the type of scan you performed.
 - **Ports/Hosts** - This tab will show the results of your port scan, including the services for those ports.
 - **Topology** - This shows the traceroute for the scan you performed. You can see how many hops your data goes through to reach the target.
 - **Host Details** - This shows a summary of your target learned through scans, such as the number of ports, IP addresses, hostnames, operating systems, and more.
 - **Scans** - This tab stores the commands of your previously-run scans. This allows you to quickly re-scan with a specific set of parameters.

12. Operating System Detection using Nmap

Nmap, short for Network Mapper, is a network discovery and security auditing tool. It is known for its simple and easy to remember flags that provide powerful scanning options. Nmap is widely used by network administrators to scan for:

- Open ports and services
- Discover services along with their versions
- Guess the operating system running on a target machine
- Get accurate packet routes till the target machine
- Monitoring hosts

Nmap Scan Types

A variety of scans can be performed using Nmap. Below are the types of scans:

TCP SCAN

A TCP scan is generally used to check and complete a three-way handshake between you and a chosen target system. A TCP scan is generally very noisy and can be detected with almost little to no effort. This is "noisy" because the services can log the sender IP address and might trigger Intrusion Detection Systems.

UDP SCAN

UDP scans are used to check whether there is any UDP port up and listening for incoming requests on the target machine. Unlike TCP, UDP has no mechanism to respond with a positive acknowledgment, so there is always a chance for a false positive in the scan results. However, UDP scans are used to reveal Trojan horses that might be running on UDP ports or even reveal hidden RPC services.

SYN SCAN

This is another form of TCP scan. The difference is unlike a normal TCP scan, nmap itself crafts a syn packet, which is the first packet that is sent to establish a TCP connection. The connection is never formed, rather the responses to these specially crafted packets are analyzed by Nmap to produce scan results.

ACK SCAN

ACK scans are used to determine whether a particular port is filtered or not. This proves to be extremely helpful when trying to probe for firewalls and their existing set of rules. Simple packet filtering will allow established connections (packets with the ACK bit set), whereas a more sophisticated stateful firewall might not.

FIN SCAN

Also a stealthy scan, like the SYN scan, but sends a TCP FIN packet instead. Most but not all computers will send an RST packet (reset packet) back if they get this input, so the FIN scan can show false positives and negatives, but it may get under the radar of some IDS programs and other countermeasures.

NULL SCAN

Null scans are extremely stealthy scan and what they do is as the name suggests — they set all the header fields to null. Generally, this is not a valid packet and a few targets will not know how to deal with such a packet. Such targets are generally some version of windows and scanning them with NULL packets may end up producing unreliable results.

XMAS SCAN

Just like null scans, these are also stealthy in nature. Computers running windows will not respond to Xmas scans due to the way their TCP stack is implemented. The scan derives its name from the set of flags that are turned on within the packet that is sent out for scanning. XMAS scans are used to manipulate the PSH, URG and FIN flags that can be found in the TCP header.

RPC SCAN

RPC scans are used to discover machines that respond to Remote Procedure Call services (RPC). RPC allows commands to be run on a certain machine remotely, under a certain set of connections. RPC service can run on an array of different ports, hence, it becomes hard to infer from a normal scan whether RPC services are running or not.

IDLE SCAN

IDLE scan is the stealthiest of all scans discussed in this nmap tutorial, as the packets are bounced off an external host. Control over the host is generally not necessary, but the host needs to meet a specific set of conditions. It is one of the more controversial options in Nmap since it only has a use for malicious attacks.

Nmap Commands

Below are the various commands you can use in Nmap along with their flag and usage description.

Scanning Techniques

Flag	Use	Example
-sS	TCP syn port scan	nmap -sS 192.168.1.1
-sT	TCP connect port scan	nmap -sT 192.168.1.1
--sU	UDP port scan	nmap --sU 192.168.1.1
--sA	TCP ack port scan	nmap --sA 192.168.1.1

Host Discovery

Flag	Use	Example
-Pn	only port scan	nmap -Pn 192.168.1.1
-sn	only host discover	nmap -sn 192.168.1.1
-PR	arp discovery on local net	nmap -PR 192.168.1.1
-n	disable DNS resolution	nmap -n 192.168.1.1

Port Specification

Flag	Use	Example
-p	specify a port/port range	nmap -p 1-30 192.168.1.1
-p-	scan all ports	nmap -p- 192.168.1.1
-F	fast port scan	nmap -F 192.168.1.1

Service Version and OS Detection

Flag	Use	Example
-sV	detect version of services	nmap -sV 192.168.1.1
-A	aggressive scan	nmap -A 192.168.1.1
-O	detect operating system	nmap -O 192.168.1.1

Timing and Performance

Flag	Use	Example
-T0	paranoid IDS evasion	nmap -T0 192.168.1.1
-T1	sneaky IDS evasion	nmap -T1 192.168.1.1
-T2	polite IDS evasion	nmap -T2 192.168.1.1
-T3	normal IDS evasion	nmap -T3 192.168.1.1
-T4	aggressive speed scan	nmap -T4 192.168.1.1
-T5	insane speed scan	nmap -T5 192.168.1.1

NSE Scripts

Flag	Use	Example
-sC	default script scan	nmap -sC 192.168.1.1
--script banner	banner grabbing	nmap --script banner 192.168.1.1

13. Do the following using NS2 Simulator

- i. NS2 Simulator - Introduction
- ii. Simulate to Find the Number of Packets Dropped
- iii. Simulate to Find the Number of Packets Dropped by TCP/UDP
- iv. Simulate to Find the Number of Packets Dropped due to Congestion
- v. Simulate to Compare Data Rate & Throughput

i. Simulation - Introduction

Network simulation is an important tool in developing, testing and evaluating network protocols. Simulation can be used without the target physical hardware, making it economical and practical for almost any scale of network topology and setup. It is possible to simulate a link of any bandwidth and delay, even if such a link is currently impossible in the real world. With simulation, it is possible to set each simulated node to use any desired software. This means that deploying software is not an issue. Results are also easier to obtain and analyze, because extracting information from important points in the simulated network is done by simply parsing the generated trace files.

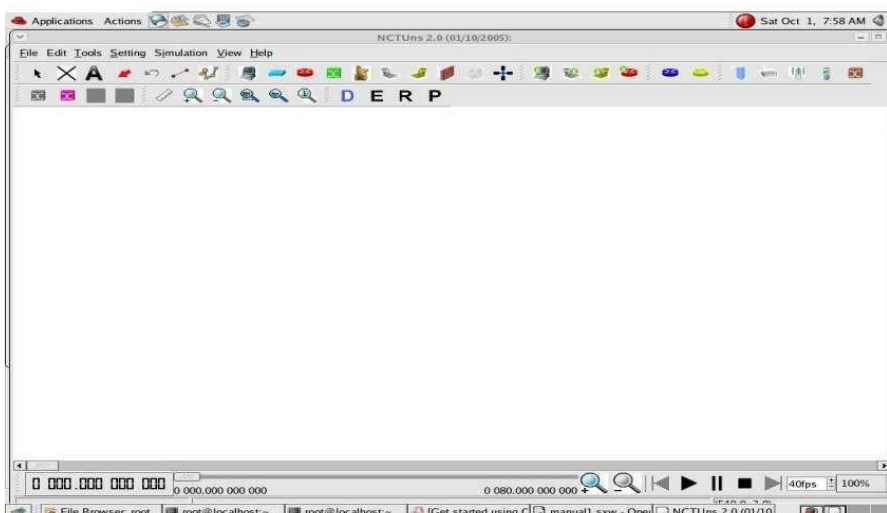
Simulation is only of use if the results are accurate, an inaccurate simulator is not useful at all. Most network simulators use abstractions of network protocols, rather than the real thing, making their results less convincing.

Setting up the Environment

A user using the NCTUns in single machine mode, needs to do the following steps before he/she starts the GUI program:

1. Set up environment variables: Before the user can run up the dispatcher, coordinator, or NCTUns GUI program he/she must set up the NCTUNSHOME environment variable.
2. Start up the dispatcher on terminal 1.
3. Start up the coordinator on terminal 2.
4. Start up the NCTUns client on terminal 3.

After the above steps are followed, the starting screen of NCTUns disappears and the user is presented with the working window as shown below:



Drawing A Network Topology

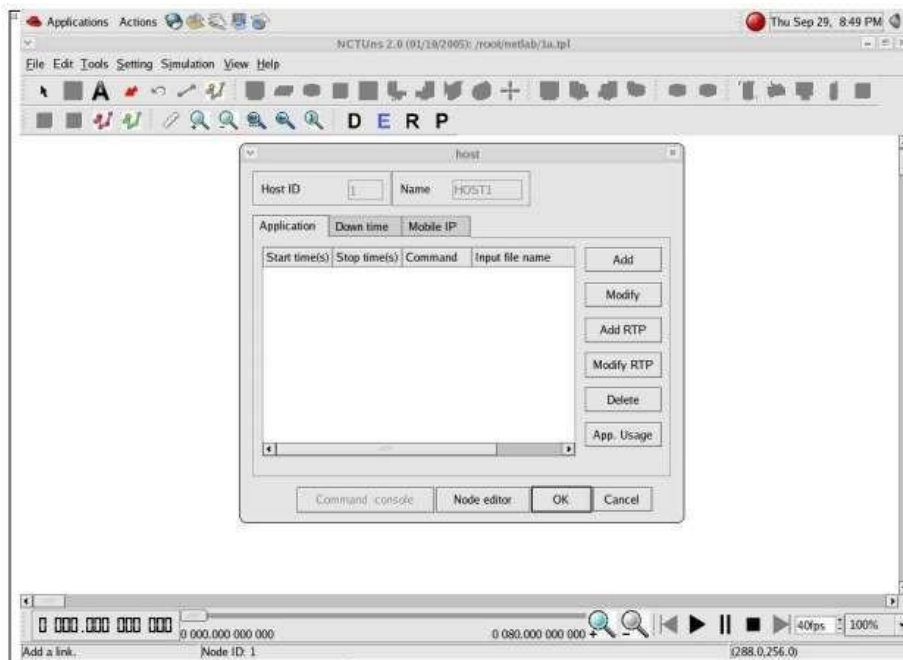
To draw a new network topology, a user can perform the following steps:

Choose Menu -> File -> Operating Mode -> and make sure that the "Draw Topology" mode is checked. This is the default mode of NCTUns when it is launched.

1. Move the cursor to the toolbar.
2. Left-Click the router icon on the toolbar.
3. Left-Click anywhere in the blank working area to add a router to the current network topology. In the same way we can add switch, hub, WLAN access point, WLAN mobile node, wall (wireless signal obstacle) etc.
4. Left-Click the host icon on the toolbar. Like in step 4, add the required number of hosts to the current topology.
5. To add links between the hosts and the router, left-click the link icon on the toolbar to select it.
6. Left-Click a host and hold the mouse button. Drag this link to the router and then release the mouse left button on top of the router. Now a link between the selected host and the router has been created.
7. Add the other, required number of links in the same way. This completes the creation of a simple network topology.
8. Save this network topology by choosing Menu -> File -> Save. It is saved with a .tpl extension.
9. Take the snapshot of the above topology.

Editing Node's Properties

1. A network node (device) may have many parameters to set. For example, we may have to set the maximum bandwidth, maximum queue size etc to be used in a network interface.
2. Before a user can start editing the properties of a node, he/she should switch the mode from the "Draw Topology" to "Edit Property" mode. In this mode, topology changes can no longer be made.
3. The GUI automatically finds subnets in a network and generates and assigns IP and MAC addresses to layer 3 network interfaces.
4. A user should be aware that if he/she switches the mode back to the "Draw Topology" mode when he/she again switches the mode back to the "Edit Topology" mode, node's IP and MAC addresses will be regenerated.



Therefore the application programs now may use wrong IP addresses to communicate with their partners.

1. In order to do so, the user must switch mode explicitly from "Edit Property" to "Run Simulation". Entering this mode indicates that no more changes can (should) be made to the simulation case.
2. Whenever the mode is switched to the "Run Simulation" mode, the many simulation files that collectively

describe the simulation case will be exported. These simulation files will be transferred to the (either remote or local) simulation server for it to execute the simulation.

Playing Back the Packet Animation Trace

After the simulation is finished, the simulation server will send back the simulation result files to the GUI program.

1. These files include a packet animation trace file and all performance log files that the user specifies to generate. Outputting these performance log files can be specified by checking some output options in some protocol modules in the node editor.
2. The packet animation trace file can be replayed later by the packet animation player. The performance curve of these log files can be plotted by the performance monitor.

Post Analysis

1. When the user wants to review the simulation results of a simulation case that has been finished before, he/she can run up the GUI program again and then open the case's topology file.
2. The user can switch the mode directly to the "Play Back" mode. The GUI program will then automatically reload the results (including the packet animation trace file and performance log file).
3. After the loading process is finished, the user can use the control buttons located at the bottom of the screen to view the animation.

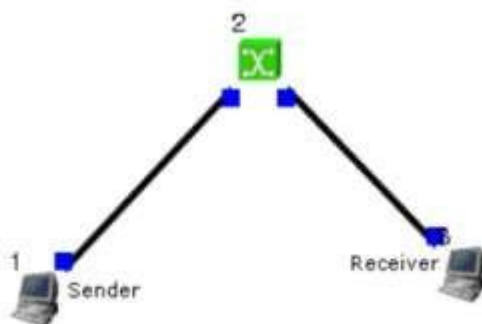
Simulation Commands

- **Run:** Start to run simulation.
- **Pause:** Pause the currently-running simulation.
- **Continue:** Continue the simulation that was just paused.
- **Stop:** Stop the currently-running simulation.
- **Abort:** Abort the currently running simulation. The difference between "stop" and "abort" is that a stopped simulation job's partial results will be transferred back to GUI files.
- **Reconnect:** The Reconnect command can be executed to reconnect to a simulation job that was previously disconnected.
- **Disconnect:** Disconnect the GUI from the currently running simulation job. The GUI now can be used to service another simulation job.

ii. Simulate to Find the Number of Packets Dropped

AIM:

Simulate a three-node point-to-point network with a duplex link between them. Set the queue size and vary the bandwidth and find the number of packets dropped.



Sender:- `step --p 2000 --l 1024 1.0.1.2`

Receiver:- `rtcp --p 2000 --l 1024`

Parameters:- Drop Packets and Collision Packets.

Step 1: Drawing Topology

1. Select/click the HOST icon on the toolbar and click the left mouse button on the editor, to place a HOST1 on the editor. Repeat the above procedure and place another host "HOST2" on the editor.
2. Select/click the HUB icon on the toolbar and click the left mouse button on the editor, to place HUB1 on the editor.
3. Click on the LINK icon on the toolbar and connect HOST1 to HUB1 and HUB1 to HOST2.
4. Click on the "E" icon on the toolbar to save the current topology. e.g: file1.tpl

NOTE: Changes cannot / (should not) be done after selecting the "E" icon

Step 2: Configuration

1. Double click the left mouse button while cursor is on HOST1 to open the HOST window.
2. Select Add button on the HOST window to invoke the command window and provide the following command in the command textbox:
`stg --u 1024 100 1.0.1.2`
3. Click OK button on the command window to exit and once again click on the OK button on the HOST window to exit.
4. Double click the left mouse button while cursor is on HOST2 to open the HOST window.
5. Select Add button on the HOST window to invoke the command window and provide the following command:
`rtg --u --w log1`
6. Click OK button on the command window to exit.
7. Click NODE EDITOR Button on the HOST window and select the MAC tab from the modal window that pops up.
8. Select LOG STATISTICS and select checkboxes for Number of Drop Packet and Number of Collisions in the MAC window.
9. Click OK button on the MAC window to exit and once again click on the OK button on the HOST window to exit.

Note: To set QUEUE size

1. Double click the left mouse button while cursor is on HOST2 to open the HOST window.
2. Click NODE EDITOR Button on the HOST window and select the FIFO tab from the modal window that pops up.
3. Change Queue size (Default 50).
4. Click OK button on the FIFO window to exit and once again click on the OK button on the HOST window to exit.

Step 3: Simulate

- i. Click "R" icon on the tool bar.
- ii. Select Simulation in the menu bar and click/select RUN in the dropdown list to execute the simulation.
- iii. To start playback select "►" icon located at the bottom right corner of the editor.
- iv. To view results, Open up new TERMINAL window, move to file1.results folder and open collision and drop log files in separate TERMINAL window.

Caution: file1 is the hypothetical name given to this simulation.

Changing Configurations

Change 1

1. Open the above file.
2. Do not change the topology or any other configuration.
3. Select E icon on the toolbar.
4. Reduce the bandwidth at link2 by double clicking the left mouse button while cursor is on link2. (Change bandwidth on both tabs Uplink/Downlink)
5. Repeat Step3 (Simulate).

Change 2

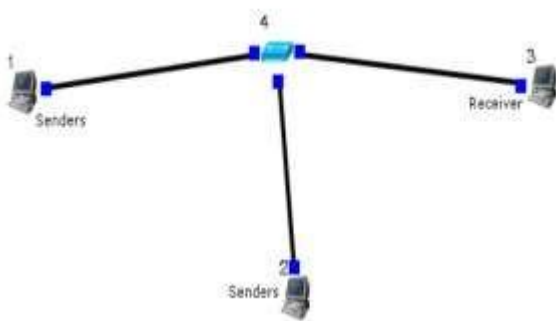
1. Open the above file.
2. Remove HUB and replace it with SWITCH.
3. Do not change anything in the configuration. Repeat Step3 (Simulate).

iii. Simulate to Find the Number of Packets Dropped by TCP/UDP

AIM:

Simulate a four-node point-to-point network and connect the link as follows: Apply a TCP agent between n0 to n3 and apply a UDP agent between n1 and n3. Apply relevant applications over TCP and UDP agents changing the parameters and determine the number of packets sent by two agents.

Topology:-



Sender:- `stcp --p 3000 --l 1024 1.0.1.3`

`stg --u 1024 1.0.1.3`

Receiver:- `rtcp --p 3000 --l 1024`

`rtg --u 3000`

Parameters:- Throughput of incoming and outgoing Packets

Step 1: Drawing Topology

1. Select/click the HOST icon on the toolbar and click the left mouse button on the editor, to place a host on the editor. Repeat the above procedure and place two other hosts "HOST2" and "HOST3" on the editor.
2. Select/click the HUB (or SWITCH) icon on the toolbar and click the left mouse button on the editor, to place a HUB (or SWITCH) on the editor.
3. Click on the LINK icon on the toolbar and connect HOST1 to HUB, HOST2 to HUB and HUB to HOST3.
4. Click on the "E" icon on the toolbar to save the current topology e.g: file2.tpl

NOTE: Changes cannot / (should not) be done after selecting the "E" icon.

Step 2: Configuration

1. Double click the left mouse button while cursor is on HOST1 to open the HOST window.
2. Select Add button on the HOST window to invoke the command window and provide the following command:
`stcp --p 21 --l 1024 1.0.1.3`
3. Click OK button on the command window to exit.
4. Click NODE EDITOR Button on the HOST window and select the MAC tab from the modal window that pops up.
5. Select LOG STATISTICS and select checkbox for output throughput in the MAC window.
6. Click OK button on the MAC window to exit and once again click on the OK button on the HOST window to exit.
7. Double click the left mouse button while cursor is on HOST2 to open the HOST window.
8. Select Add button on the HOST window and provide the following command:
`stg --u 1024 100 1.0.1.3`
9. Click OK button on the command window to exit.
10. Click NODE EDITOR Button on the HOST window and select the MAC tab.
11. Select LOG STATISTICS and select checkbox for output throughput in the MAC window.
12. Click OK button on the MAC window to exit and once again click on the OK button on the HOST window to exit.
13. Double click the left mouse button while cursor is on HOST3 to open the HOST window.
14. Select Add button on the HOST window and provide the following command:
`rtcp --p 21 --l 1024`
15. Click OK button on the command window to exit.
16. Also add the following command on HOST3:
`rtg --u --w log1`
17. Click NODE EDITOR Button on the HOST window and select the MAC tab.
18. Select LOG STATISTICS and select checkbox for input and output throughput in the MAC window.
19. Click OK button on the MAC window to exit and once again click on the OK button on the HOST window to exit.

Step 3: Simulate

- i. Click "R" icon on the tool bar.
- ii. Select Simulation in the menu bar and click/select RUN in the dropdown list to execute the simulation.
- iii. To start playback select "►" icon located at the bottom right corner of the editor.
- iv. To view results, Open up new TERMINAL window, move to file2.results folder and open input and output throughput log files in separate TERMINAL window.

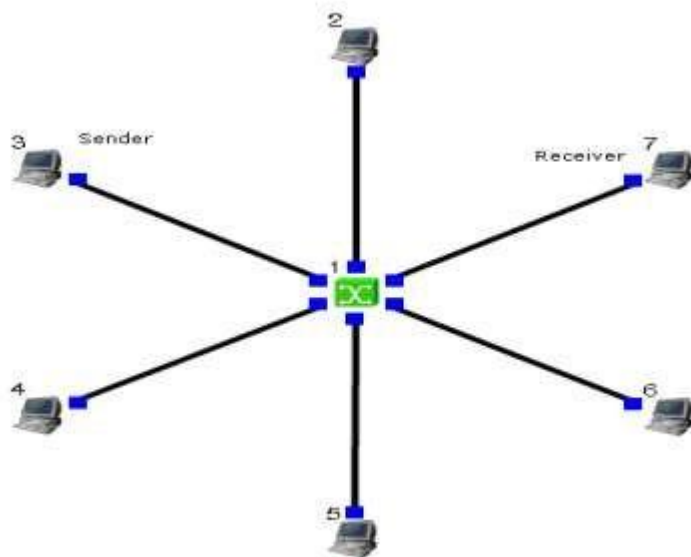
Caution: file2 is the hypothetical name given to this simulation.

iv. Simulate to Find the Number of Packets Dropped due to Congestion

AIM:

Simulate the transmission of ping messages over a network topology consisting of 6 nodes and find the number of packets dropped due to congestion.

Topology:-



Sender:- `step --p 2000 --l 1024 1.0.1.4`

Receiver:- `rtcp --p 2000 --l 1024`

Command Console:- Goto tools -> simulation time and change Simulation time to 100. During run mode, double click host 2 and then click command console. And execute:

`ping 1.0.1.4`

Parameters:- Drop Packets and Collision Packets.

Step 1: Drawing Topology

1. Select/click the SUBNET icon on the toolbar and click the left mouse button on the editor, to place a SUBNET on the editor.
2. A pop up window appears requesting the number of nodes and radius for the subnet. Set number of nodes=6; Set radius of subnet >150.
3. Click on the "E" icon on the toolbar to save the current topology e.g: file4.tpl

NOTE: Changes cannot / (should not) be done after selecting the "E" icon.

Step 2: Configuration

1. Double click the left mouse button while cursor is on a HOST to open the HOST window.
2. Click NODE EDITOR Button on the HOST window and select the INTERFACE tab (1st tab) from the modal window that pops up.
3. Determine the IP address of the selected host.
4. Click OK button on the INTERFACE window to exit and once again click on the OK button on the HOST window to exit.
5. Repeat the above step for 2 other HOSTS.
6. Also click NODE EDITOR Button on the HOST window and select the MAC tab from the modal window that pops up.
7. Select LOG STATISTICS and select checkbox for drop and collision log statistics in the MAC window.
8. Click OK button on the MAC window to exit and once again click on the OK button on the HOST window to exit.
9. Repeat steps 6 to 9 for the other hosts selected at step 5.
10. Select G_Setting from the menu bar and select Simulation from the drop down list. Set simulation time > 600sec.

Step 3: Simulate

- i. Click "R" icon on the tool bar.
- ii. Select Simulation in the menu bar and click/select RUN in the dropdown list to execute the simulation.
- iii. During simulation, open a new terminal window.
- iv. Type ping IP address of a host in the subnet at the command prompt.
- v. To view results, Open up new TERMINAL window, move to file4.results folder and open drop and collision log files in separate TERMINAL window.

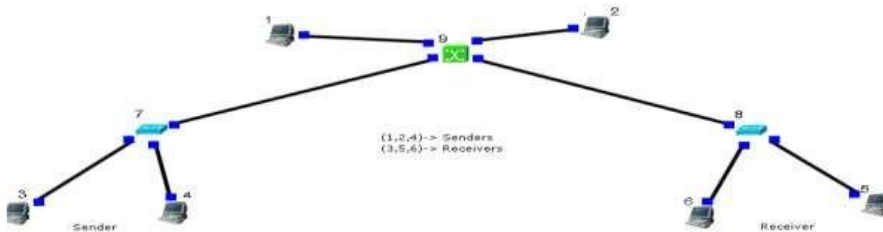
Caution: file4 is the hypothetical name given to this simulation.

v. Simulate to Compare Data Rate & Throughput

AIM:

Simulate an Ethernet LAN using N nodes (6-10), change error rate and data rate and compare throughput.

Topology:-



Sender:- `stcp --p 2000 --l 1024 1.0.1.4`

Receiver:- `rtcp --p 2000 --l 1024`

Double click on receiver link and change BER to 0.000001, Run Again.

Parameters:- Throughput of outgoing Packets

Step 1: Drawing Topology

1. Select/click the HOST icon on the toolbar and click the left mouse button on the editor, to place HOST1 on the editor. Repeat the above procedure and place 5 other hosts "HOST2", "HOST3", "HOST4", "HOST5", and "HOST6" on the editor.
2. Select/click the HUB icon on the toolbar and click the left mouse button on the editor, to place HUB1 on the editor. Repeat the above procedure and place another HUB2 on the editor.
3. Click on the LINK icon on the toolbar and connect HOST1, HOST2 and HOST3 to HUB1, HOST4, HOST5 and HOST6 to HUB2.
4. Select/click the SWITCH icon on the toolbar and click the left mouse button on the editor, to place SWITCH1 on the editor.
5. Click on the LINK icon on the toolbar and connect HUB1 to SWITCH1 and HUB2 to SWITCH1.
6. Click on the "E" icon on the toolbar to save the current topology e.g: file5.tpl

NOTE: Changes cannot / (should not) be done after selecting the "E" icon.

Step 2: Configuration

1. Double click the left mouse button while cursor is on HOST1 to open the HOST window.
2. Select Add button on the HOST window and provide the following command:
`stcp --p 21 --l 1024 1.0.1.4`
3. Click OK button on the command window to exit and once again click on the OK button on the HOST window to exit.
4. Repeat this step at HOST 2 and HOST3, but use different commands:

```
stcp --p 21 --l 1024 1.0.1.5    at HOST2
stcp --p 21 --l 1024 1.0.1.6    at HOST3
```

5. Double click the left mouse button while cursor is on HOST4 to open the HOST window.
6. Select Add button on the HOST window and provide the following command:

```
rtcp --p 21 --l 1024
```
7. Click OK button on the command window to exit.
8. Click NODE EDITOR Button on the HOST window and select the MAC tab.
9. Select LOG STATISTICS and select checkbox for output throughput in the MAC window.
10. Click OK button on the MAC window to exit and once again click on the OK button on the HOST window to exit.
11. Repeat this step at HOST 5 and HOST6:

```
rtcp --p 21 --l 1024    at HOST5
rtcp --p 21 --l 1024    at HOST6
```
12. Double click the left mouse button while cursor is on HOST5 to open the HOST window.
13. Click NODE EDITOR Button on the HOST5 window and select the PHYSICAL tab.
14. Change Bit Error Rate.
15. Click OK button on the PHYSICAL window to exit and once again click on the OK button to return to the HOST window.
16. Click NODE EDITOR Button on the HOST window and select the MAC tab.
17. Select LOG STATISTICS and select checkbox for output throughput in the MAC window.
18. Click OK button on the MAC window to exit and once again click on the OK button on the HOST window to exit.
19. Repeat this step for HOST6. Change Bandwidth this time while undoing the change in Bit Error Rate, also select the output throughput at HOST6.

Step 3: Simulate

- i. Click "R" icon on the tool bar.
- ii. Select Simulation in the menu bar and click/select RUN in the dropdown list to execute the simulation.
- iii. To start playback select "▶" icon located at the bottom right corner.
- iv. To view results, Open up new TERMINAL window, move to file5.results folder and open output throughput log files in separate TERMINAL window.

Caution: file5 is the hypothetical name we gave to this simulation.

Viva Questions

1. What are the 7 layers of the OSI model?
2. Difference between OSI model and TCP/IP model?
3. Difference between TCP and UDP?
4. What is subnetting and why is it used?
5. Difference between hub, switch, and router?
6. What is ARP and how does it work?
7. What is DNS and its purpose?
8. Difference between circuit switching and packet switching?
9. What is the three-way handshake in TCP?
10. Difference between static and dynamic routing?
11. Character strings used for start/end of frame in character stuffing?
12. Bit stuffing output for input frame 11111100?
13. Full form of CRC and polynomials used?
14. Significance of window size in sliding window protocol?
15. How does Go-Back-N handle loss recovery?
16. Primary objective of Dijkstra's algorithm?
17. Basic principle of Distance Vector Routing?
18. What network issue does Leaky Bucket algorithm solve?
19. Leaky Bucket: variable or constant transmission rate?
20. Purpose of bit stuffing and character stuffing?